

Security Controls in Shared Source Code Repositories

Carolina Rodriguez
CSD-380
Module 10

RoadMap

- What Could Go Wrong?
- Control access -Make Sure the Right People Have Access
- Protect Important Branches -Add Checks Before Code Is Merged
- Scan code automatically -Build Security Checks Into the Workflow
- Protect secrets and Dependencies - Keep Sensitive Information Out of the Repository
- Monitor and improve -Keep an Eye on Repository Activity

What could go wrong?

Shared repositories can expose more than just code.

- Someone gets access who should not have it
- Passwords, API keys, or tokens are accidentally committed
- Unreviewed code is merged into the main branch
- A vulnerable third-party package is added
- Source code is copied or downloaded without approval
- A developer account or device is compromised

Make Sure the Right People Have Access

Not everyone needs full access to every repository.

Ways you can control access is by:

- Require multifactor authentication
- Give users only the access they actually need
- Avoid shared accounts
- Limit administrator permissions
- Review access regularly
- Remove access quickly when someone leaves or changes roles
- Keep track of third-party apps and service accounts

Protect Important Branches- Add Checks Before Code Is Merged

Important code changes should always be reviewed before they go live.

- Limit who can approve or release code
- Have at least one reviewer
- Block direct changes to the main branch.
- Require pull requests



Build Security Checks Into the Workflow

Security checks work best when they happen automatically.

You can group the tools into:

Before commit

- IDE checks for insecure code
- Pre-commit scanning to check files

During pull request

- Static application security testing (SAST) –looking for weak code
- Secret scanning for credentials or tokens
- Dependency scanning to check vulnerabilities in libraries

Before deployment

- Container scanning to check vulnerabilities
- Dynamic Application Security Testing (DAST)
- Security approval gates to hold deployment

Keep Sensitive Information Out of the Repository

A repository can be secure while still depending on an unsafe package or exposed credential.

Secrets

- Do not store passwords or API keys in code
- Use a secrets manager
- Scan repository history for exposed credentials
- Rotate secrets quickly if they are leaked
- Avoid placing sensitive data in logs

Dependencies

- Use approved package sources
- Pin dependency versions
- Keep lock files
- Scan for known vulnerabilities
- Remove unused packages
- Review major dependency updates

Keep an Eye on Repository Activity

Repository security is not a one-time setup. It needs regular checking and improvement like:

- Monitoring logins, downloads, cloning, and admin changes
- Watching for unusual activity
- Checking whether branch protections are still enabled
- Tracking exposed secrets and vulnerable dependencies
- Reviewing inactive users and unused accounts

A roadmap can look like:

Start

Require Multi-Factor authentication
Review repository permissions
Protect the main branch

Next

Add automated scanning
Set up approval rules
Improve secrets management

Overtime

Monitor repository health
Review alerts
Update controls as risks change

Berecki, B. (2025, November 20). *Best practices for source code security*. Endpoint Protector.
<https://www.endpointprotector.com/blog/your-ultimate-guide-to-source-code-protection/>

Ghalieb, Z. (2024, October 31). *Source code security: Basics and best practices*. Wiz.
<https://www.wiz.io/academy/application-security/source-code-security>

Kusari. (n.d.). *Version control security*.
<https://www.kusari.dev/learning-center/version-control-security>

OpenAI. (2026). *ChatGPT* [Large language model].
<https://chatgpt.com/>

Rose, J. (2025, June 11). *Repository health monitoring part 2: Essential practices for secure repositories*. Checkmarx.
<https://checkmarx.com/blog/supply-chain-security/repository-health-monitoring-part-2-essential-practices-for-secure-repositories/>

A graphic for the 'References' section. It features a dark teal background with a large, lighter teal abstract shape. The word 'References' is written in white. On either side of the central shape are vertical stacks of horizontal white lines, resembling a bookshelf or a list of references.

References